Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer, Communication and Electronic Engineering

Final Dissertation

Distributed Rabbit Order

A Distributed Parallel Scalable Graph Reordering Algorithm for High-Performance Computing

Supervisor
Prof. Vella F.

Student
Cassone Valerio

Academic year 2024/2025

Acknowledgements

I would like to express my deepest gratitude to Professor Flavio Vella for his invaluable guidance, constant support, and insightful feedback throughout the development of this thesis. His expertise and dedication were fundamental in shaping both the direction and quality of this work. I am also sincerely thankful to Dr. Paolo Sylos Labini, whose patience, availability, and technical advice were essential during the research and implementation phases. His mentorship and encouragement greatly contributed to my growth throughout this project.

Finally, I would like to thank my colleagues and friends for their support, encouragement, and the many moments of shared challenges and achievements that made this journey more enjoyable and meaningful. A heartfelt thank you goes to my family, who never grew tired of listening to me talk endlessly about things they didn't always understand, but always supported me with patience, love, and genuine interest. Their presence has been a constant source of motivation.

Contents

Abstract

In recent decades, graph analysis has emerged as a fundamental discipline within computer science and data analytics. Its exponential growth in importance and diffusion is mainly due to the large number of applications in all scientific fields, ranging from knowledge discovery in real-world graphs (e.g. social networks and web graphs) to the analysis of biological networks. As the amount of data to be collected and processed increases, the main challenge today is no longer finding new solutions to existing problems. Instead, the focus is primarily on searching for new optimisations for existing solutions.

When using graph analysis algorithms, input data are often stored in matrices with density heavily depending on the graph structure. Therefore, poor locality of memory accesses, which derives from the unstructured and complex relationships among the entities represented by graphs, is the main cause of execution time problems [30]. To improve memory access, one possible optimisation is ahead-of-time data layout processing. This involves analysing the graph's structure and reordering it by relabelling the nodes to maximise the locality of non-zero values. Therefore, this process requires an efficient reordering algorithm that justifies its application in terms of execution time. In other words, the time spent reordering the graph must be offset by a measurable reduction in the execution time of subsequent computations. If the reordering process is too costly, it may outweigh the benefits it is intended to deliver, particularly in performance-critical or large-scale applications.

Starting from existing reordering algorithms, we analyse a community-based sparse matrix reordering algorithm, Rabbit Order [3], which comes with shared-memory parallelisation. We instead implement a new version with distributed-memory parallelisation using MPI. In order to address this challenge, our novel implementation needs to minimise the inter-process communication overhead.

After the algorithm has been implemented in C++, it was tested on the UniTrento HPC cluster using graphs of increasing dimensions in order to compute the speedup and efficiency of the application. Furthermore, the execution times of individual steps were compared to identify bottlenecks.

The test results demonstrate the application's promising performance and scalability under a range of conditions, particularly with larger graphs, with some configurations achieving more than the ideal speedup. The main bottlenecks were identified as being related to an increase in the number of processes, particularly for smaller graphs. This highlights the need for an improved partitioning system for load balancing, as well as the need to dynamically adjust the number of processes, especially during the final iterations of community detection. Furthermore, the application produced better results with sparser graphs and more balanced networks, while communication overhead was revealed to be heavier for densely connected ones.

Overall, the proposed MPI-based reordering algorithm demonstrates that Rabbit Order can be effectively extended to distributed-memory systems, paving the way for further optimisations and broader applicability in large-scale graph analytics.

1 Introduction

In graph theory, a graph (also referred to as a network) is a collection of different entities (called nodes or vertices) and connections among them (called edges or links) [37]. In a more mathematical form, a graph is an ordered triple $G = (V, E, \phi)$ comprising:

- V, a set of vertices;
- E, a set of edges;

- $\phi : E \to \{\{x, y\} \mid x, y \in V\}$, an incidence function mapping every edge to a pair of vertices.

The order of the graph is the number of vertices $|V|$, and it is assumed to be non-empty. The size of a graph is the number of edges $|E|$. An edge is considered a loop if its endpoints (the vertices it connects) are the same vertex. The degree of a vertex is the number of edges that are incident to it, where loops are counted twice. From there, the degree of a graph is the maximum of the degrees of its vertices [37].

Based on these definitions, an undirected graph is a graph in which edges don't have orientations (the edge $\{x, y\}$ indicates a connection from vertex x to vertex y and vice versa), while a directed graph is a graph in which edges have orientations (the edge $\{x, y\}$ indicates only a connection from vertex x to vertex y). Moreover, a weighted graph is a graph which has a weight (e.g. a number) associated to each edge (in case of unweighted graph the weight can be assumed to be 1).

1.1 Graph representations

Directly from the definition, a graph could be easily represented with an edge list, that is the list of all the edges of the graph, or with an adjacency list, that is a map where each vertex is associated with a list of its neighbouring vertices. These two are the most straightforward and direct ways of representing graphs that come directly from the definition. However, these are not always the best for graph analysis.

A more computation-oriented representation of a graph could be achieved by using matrices: the adjacency matrix is a square matrix which elements indicate whether pairs of vertices are connected in the graph [44]. Rows and columns are associated with each vertex. Each row of the matrix can be seen as the adjacency list of each vertex of the graph and the elements can be 0 if there is not an edge between those row and column vertices and it can be 1 (or the associated weight) otherwise. The adjacency matrix can be directly derived from the edge list or the adjacency list of a graph (assumed that the number of vertices is known a priori or derived from the maximum vertex identifier found while reading the list). For an undirected graph, the adjacency matrix will always be symmetric.

Through the matrix representation of a graph it is possible to quickly visualize the density of a graph, defined as the number of edges (size) with respect to the maximum possible edges $n \times n$. From this, a dense graph will correspond to an highly populated dense adjacency matrix and, on the contrary, a sparse graph will correspond to a sparse matrix, where the number of non-zero values is significantly lower than the size of the matrix.

When it comes to storing and manipulating a graph for computation, there exist more compact representations that can be written starting from its adjacency matrix. This reduces the amount of memory required to store it and avoid storing values for non-zero elements [45] [12].

The Compressed Sparse Row (CSR) or Compressed Row Storage (CRS) format represents a $n \times n$ matrix with three arrays: *row*, *column* and *values*. The *column* and *values* arrays have the size equal to the number of non-zero elements: the first contains the column where the non-zero value is located in the matrix and the second contains the non-zero value itself. The *row* array has $n + 1$ elements and, for each row of the matrix, it contains the starting indexes for its non-zero values *column* and *values* arrays. This means that *row*[i] is the index of the first non-zero value in the i-th row for the *column* and *values* arrays, $row[i + 1] - row[i]$ will be the number of non-zero elements in the i-th row and *row*[n] will be the number of non-zero elements of the whole matrix [6].

The Compressed Sparse Column (CSC) is similar to CSR with the difference in reading the values first by column and then in the row and values arrays. Figure 1.1 and Figure 1.2 reproduced from Kelly et al. [20] summarise the main graph representations described.

1.2 Use of adjacency matrix for analysis

Representing a graph with an adjacency matrix allows to completely change the domain of the analysis: the graph is transformed from a complex and unstructured package of data into a linear algebraic object that can be manipulated and, therefore, desired operations on it can be turned to algorithmic mathematical computations. To give an example, in machine learning systems neural networks can be considered proper graphs. In particular, graphs are at the core of a new and modern branch of deep

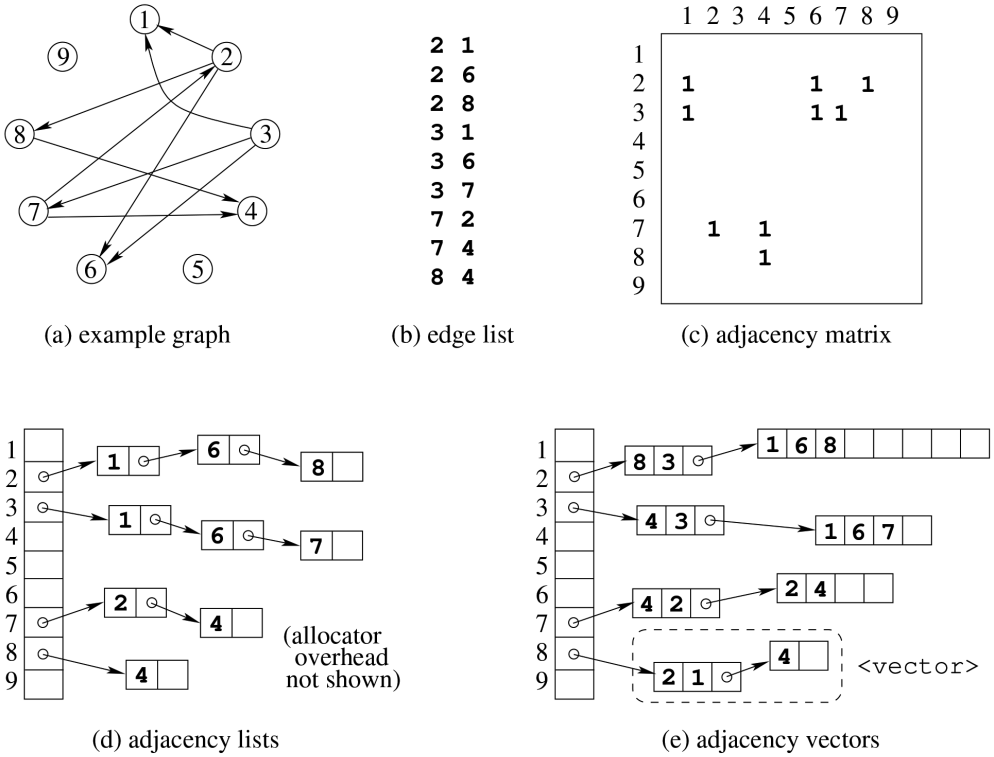

Figure 1.1: Equivalent graph representations

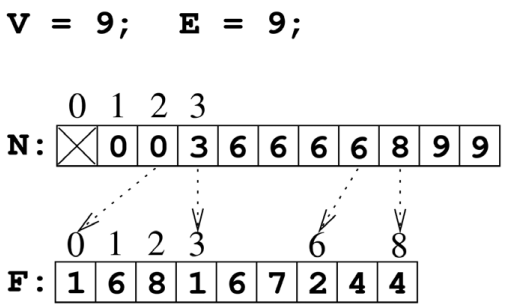

Figure 1.2: CSR representation of Figure 1.1 graph

learning, the so called Graph Neural Networks [39]. Hence, all the computations needed for training, learning and predicting, despite being done on graphs, actually result in matrices multiplication.

The General Matrix-Matrix (GEMM) product is defined as $C = \alpha AB + \gamma C$, where α and γ are scalars and A, B, C are rectangular matrices [8]. When working with graphs and adjacency matrices, the operands of the equation are sparse matrices. Hence, solving the product with the general approach will result in an underperforming computation, due to the large quantity of zeros involved in the multiplications. For this reason these operations require new strategies addressed in the Sparse Matrix-Matrix product (SpMM) field of study. Rather than developing more efficient ways for performing the product in case of sparse matrices operands, this work focuses on finding new methods to arrange data before multiplying it [47].

1.3 Challenges in graph processing

As mentioned in the previous paragraph, processing graphs can be sped up by permuting initial data before using it. The main problem to deal with in graph analysis is the poor locality of the memory accesses, which is strictly related to the innate complex structure of the graph. Imagine processing a graph as the one depicted in Figure 1.1 with its adjacency matrix. Typical graph analysis algorithms process the graph one vertex at a time, starting with vertex 1 and then moving on to its neighbours. The next step is to move to vertex 2 and again move to its neighbours. Regardless of the complexity of the computation performed by the algorithm, this approach will lead to frequent cache misses due to:

- **poor temporal locality**: each vertex is accessed only once and subsequent steps require completely different vertices every time; there's no reuse of the same nodes and so the program generates cache misses at every step;
- **poor spatial locality**: vertices that are directly connected in the graph are not co-located in memory because of their arbitrary labelling and distance in the adjacency matrix (or whatever representation used); there's no collection of multiple edges in a single fetch and so the program generates cache misses of scattered data.

One solution to this problem is to reorder the matrix, which results in a relabelling of all the nodes in the graph. This operation is harmless because it doesn't change the structure of the graph, that is it neither creates nor destroys edges, but consists in finding a permutation of the rows and the columns of the adjacency matrix of a graph. This permutation must be chosen to improve the performance of future processing applied to the graph in terms of temporal and spatial memory locality by trying to minimize cache misses and memory accesses. The two new problems to face are then reduced to finding the "best" permutation of the graph and ensuring that the time spent computing the reordering is justified by a sufficient reduction in the execution time of the algorithm, compared to running it on the original, unordered graph.

Identifying the optimal permutation is not the focus of this study. Instead, starting from a state-of-the-art algorithm, this work aims to optimize an existing solution by accelerating the execution of the algorithm. In this context the optimization is defined as **the reduction of the overall execution time** of the chosen algorithm.

1.4 Parallelisation

The most straightforward way to accelerate an algorithm is to use parallelisation, i.e. the simultaneous execution of different tasks that contribute to the same application [15]. Common abstractions proposed are task-parallelism and data-parallelism. In task parallelism, an application's workload is divided into several tasks, which are then performed by different computational units and coordinated by events. This approach is intended for complex applications whose tasks are different, independent and capable of being executed in parallel. In contrast, data parallelism involves executing the same task on different portions of data simultaneously. This approach necessitates communication and synchronisation to ensure coherence with the data being processed.

In graph analysis, data-parallelism is essential for improving the performance of algorithms in terms of execution time. Due to the large size of input graphs and the relatively simple operations performed on them, such as traversal and basic mathematical computations, achieving parallelisation and distributing the workload among different computational units can significantly improve execution time. However, graph problems have some inherent characteristics that make them unsuitable for current computational problem-solving approaches. In particular, as described by Lumsdaine et al. [30], the following properties of graph problems pose a significant challenge to achieving efficient parallelism:

- **Data-driven computations**: Graph computations are often entirely data-driven. The computations performed by a graph algorithm depend on the structure of the graph's vertices and edges (nodes and links) rather than being directly expressed in code. Consequently, expressing parallelism based on the partitioning of computation can be challenging because the structure of the computations within the algorithm is not known in advance.
- **Unstructured problems**: The data in graph problems is usually unstructured and highly irregular. Similar to the difficulties encountered when parallelising a graph problem based on its computational structure, the irregular structure of graph data makes it challenging to identify opportunities for parallel processing by partitioning the data. Poorly partitioned data can result in unbalanced computational loads, which can limit scalability.
- **Poor locality**: As graphs represent relationships between entities that may be irregular and unstructured, computations and data access patterns tend not to be very localised. This is particularly true for graphs derived from data analysis. The performance of contemporary processors relies on exploiting locality. Therefore, achieving high performance with graph algorithms can be challenging, even on serial machines.
- **High data access to computation ratio**: Graph algorithms often prioritise exploring the structure of a graph over performing large-scale computations on graph data. Consequently, the ratio of data access to computation is higher than for scientific computing applications. Since these accesses tend to have little exploitable locality, the runtime can be dominated by waiting for memory fetches.

Despite these challenges, the performance benefits of parallelisation make it a worthwhile pursuit in graph analysis. The aim of this thesis is therefore to investigate and implement a parallel version of an existing graph algorithm to address the aforementioned difficulties and improve its execution efficiency on large-scale graphs.

2 Related Work

Before analysing and parallelising an existing algorithm, it is necessary to gain an understanding of the current solutions for sparse matrix reordering, in order to identify a suitable approach for parallelisation.

2.1 Approximate Minimum Degree

The Approximate Minimum Degree (AMD) [2] is a heuristic algorithm used to reorder the rows and columns of a sparse matrix to reduce fill-in during factorization (e.g., LU or Cholesky). It approximates the minimum degree ordering by selecting nodes with the lowest degree in a graph representation of the matrix, where nodes represent rows/columns and edges represent non-zero entries. At each step, it selects the node with the smallest approximate degree and eliminates it, updating the graph to reflect the fill-in created by this elimination. The degree is only approximately computed to reduce cost, using techniques like mass elimination and aggressive absorption to avoid redundant work. This

continues until all nodes are eliminated, producing a permutation that tends to minimize fill-in. AMD is computationally efficient and often produces good results in practice, significantly improving the performance of direct solvers.

2.2 Nested Dissection

Nested Dissection (ND) [13] is a graph-based ordering algorithm for sparse matrices that aims to minimize fill-in during matrix factorization. It recursively partitions the graph of the matrix into smaller subgraphs by removing a small set of nodes (a separator) whose removal disconnects the graph. The separator is computed with graph partitioning techniques. The nodes in the separator are ordered last, and the process is applied recursively to the remaining subgraphs. This hierarchical ordering effectively reduces the bandwidth and fill-in, especially for matrices arising from grid-like problems.

2.3 Reverse Cuthill-McKee

Reverse Cuthill-McKee (RCM) [7] is a heuristic algorithm designed to reduce the bandwidth of sparse symmetric matrices. The bandwidth corresponds to how far non-zero elements are from the diagonal, and reducing it can lead to better performance in both storage and computation, especially in iterative solvers or when exploiting locality of reference. The algorithm works by interpreting the matrix as an undirected graph, where each node represents a row (or column) and edges represent non-zero entries. It performs a breadth-first traversal starting from a node with low degree, visiting nodes level by level. Within each level, nodes are sorted by increasing degree to encourage tighter grouping around the diagonal. Once the traversal is complete, the resulting ordering is reversed—hence the name Reverse Cuthill-McKee. This reversal tends to place high-degree nodes near the centre of the matrix, which empirically leads to better bandwidth reduction than the original Cuthill-McKee order. Although RCM does not aim to reduce fill-in for direct solvers like AMD or Nested Dissection, it remains useful in applications where a narrow bandwidth is beneficial, and it is valued for its simplicity and speed.

2.4 Spectral Ordering

Spectral Ordering [16] [10] is a reordering technique for sparse symmetric matrices based on spectral graph theory. It aims to reduce bandwidth or improve locality in the matrix structure by exploiting the eigenstructure of the graph associated with the matrix. The method is particularly effective in clustering well-connected nodes together in the final ordering. The process begins by constructing the Laplacian matrix of the graph $L = D - A$, where A is the adjacency matrix and D is the diagonal matrix of node degrees. The Laplacian encapsulates the connectivity of the graph and serves as the foundation for spectral analysis. Next, the algorithm computes the Fiedler vector, which is the eigenvector corresponding to the second smallest eigenvalue of the Laplacian. This vector contains valuable information about the graph's structure: sorting the nodes based on its values often results in a natural ordering where strongly connected nodes are placed near each other. Finally, the nodes of the graph are ordered according to the ascending or descending values of the Fiedler vector. This typically leads to a reduction in the matrix bandwidth and improved performance of direct or iterative solvers.

2.5 Graph Partitioning

Graph Partitioning [19] is a widely used strategy for reordering sparse matrices, particularly in the context of parallel computing and direct solvers. The main objective is to divide the graph corresponding to a sparse matrix into smaller, roughly equal-sized subgraphs while minimizing the number of edges between them. This reordering helps to reduce fill-in during factorization and enables efficient parallel processing by localizing computations. The goal is to split the graph into k parts (partitions) such that each part contains approximately the same number of nodes, and the number of edges crossing between parts (the edge cut) is minimized. Once the graph is partitioned, a reordering of the matrix is derived by grouping nodes belonging to the same partition together. This structure tends to

cluster non-zero entries along the diagonal blocks of the matrix, improving data locality and reducing inter-process communication in distributed memory settings. Graph partitioning is often implemented using multilevel algorithms. These algorithms proceed in three stages: coarsening, where the graph is progressively reduced to a smaller one; initial partitioning of the coarse graph; and uncoarsening, where the partitioning is refined as the graph is expanded back to its original size. This approach strikes a good balance between partitioning quality and computational cost.

A widely used tool that implements this approach is METIS [18], a software library developed by Karypis and Kumar. METIS provides efficient algorithms for graph partitioning, producing high-quality partitions with low edge cuts in relatively short computation time. It also supports node reordering based on partitioning, making it a practical solution for reducing fill-in in sparse matrix factorization.

2.6 Space-Filling Curves

Space-Filling Curves (SFC) [4] are a class of methods used for reordering sparse matrices based on geometric information, particularly when the matrix originates from the discretization of physical domains, such as in finite element or finite difference methods. The core idea is to map the multi-dimensional coordinates of the matrix's unknowns (or nodes) into a one-dimensional sequence that preserves locality — that is, points that are close in space remain close in the ordering. Among the most common space-filling curves are the Hilbert curve, the Z-order (Morton curve), and the Peano curve. These curves traverse a multidimensional domain in a continuous path that touches every point (or grid cell) without crossing itself, thereby providing a linear ordering that approximates the spatial proximity of the nodes.

The algorithm typically works in the following way: each node of the computational mesh is assigned a position in space and this position is mapped to a scalar SFC index using the chosen curve (e.g., Hilbert indexing). The nodes are then sorted by this index, and the resulting ordering is used to permute the rows and columns of the sparse matrix. This reordering tends to cluster non-zero entries near the diagonal, improve memory access patterns, and enhance cache utilization, especially in matrix-vector operations. While SFCs do not directly aim to minimize fill-in during factorization, they are particularly effective in improving performance for iterative solvers, parallel load balancing, and sparse matrix storage formats.

3 Parallel Reordering

After an overview over the existing sparse matrix reordering algorithms, we chose to focus on the Rabbit Order algorithm, by designing a new implementation introducing parallelisation for distributed-memory systems.

3.1 Rabbit Order

In 2016 Arai et al. proposed Rabbit Order [3], a just-in-time parallel reordering algorithm for fast graph analysis. As with other reordering algorithm, the aim of Rabbit Order is to preprocess a graph in order to find a permutation that improves the performance of subsequent analysis, in particular for SpMV operations. The key strength of the Rabbit Order algorithm lies in its scalability and efficient parallelization capabilities, which allow it to handle large-scale sparse matrices effectively. Algorithm 1 shows the overview of Rabbit Order as it is presented in the reference paper. The main steps of the algorithm are two: the community detection to identify the partition and the ordering generation.

3.1.1 Overview of the algorithm

Community detection is the key point of the algorithm: a community is a group of graph nodes that "belong together" according to some precisely defined criteria which can be measured [11]. There are, however, several definitions of communities. A common approach is to define a community as a group

Algorithm 1: Overview of Rabbit Order

Input : Graph $G = (V, E)$
Output: Permutation $\pi : V \to \mathbb{N}$ for new nodes ordering
// Perform hierarchical community-based ordering
dendrogram $\leftarrow$ CommunityDetection();
return OrderingGeneration(*dendrogram*);
Function CommunityDetection()**:**
    // Perform incremental aggregation
    foreach $u \in V$ *in increasing order of degree* **do**
        $v \leftarrow$ neighbour of u that maximizes $\Delta Q(u, v)$;
        if $\Delta Q(u, v) > 0$ **then**
            Merge u into v and record this merge in dendrogram;
    return *dendrogram*;
Function OrderingGeneration(*dendrogram*)**:**
    $new\_id \leftarrow 0$;
    foreach $v \in V$ *in DFS visiting order on dendrogram* **do**
        $\pi[v] \leftarrow new\_id$;
        $new\_id \leftarrow new\_id + 1$;
    return π;

of nodes such that the density of edges between nodes of the group is higher than the average edge density in the graph. Based on this definition, it is possible to numerically quantify the quality of a particular division of a network by using modularity.

Modularity Q has been firstly defined in 2004 by Newman and Girvan [34] and quickly became widely used in subsequent works as a quality measure of a partition of a graph. One of the operational definition of modularity is [5]:

$$Q = \frac{1}{2m} \sum_{u,v} \left(A_{uv} - \frac{k_u k_v}{2m} \right) \delta(c_u, c_v) \tag{3.1}$$

In the equation, $2m$ is the total weight of the graph counted twice for undirected graphs (or the total number of edges in case of unweighted graphs), A is the adjacency matrix associated with the graph, hence A_{ij} is the weight of the edge between nodes u and v, k_u is the weighted degree of node u, hence the sum of the weights of the edges attached to node u, c_u is the community index assigned to node u and $\delta(c_u, c_v)$ is the Kronecker delta function that return 1 if $c_u = c_v$ (hence nodes u and v belong to the same community) and 0 otherwise.

Based on this definition of modularity, the Rabbit Order algorithm performs a two-step hierarchical, community-based ordering process. First, it extracts hierarchical communities in an agglomerative manner by aggregating nodes to their neighbours and simultaneously constructing a dendrogram. The second step involves converting the dendrogram into a new ordering. For each community identified, the vertices within that community are co-located.

At the start of the algorithm each node represents its own community constituted only by the node itself. At this point, in order to perform the community detection, each node must be assigned to the best community available based on the communities of its neighbours. For this reason, for the node u the algorithm consists in finding the best neighbour v of u such that the modularity of the graph increases when u is assigned to the community of v. There exists a compact equation to compute the modularity gain ΔQ of this assignment that can be directly derived from the definition of Q [41]:

$$\Delta Q = 2 \left\{ \frac{k_{u,c_v}}{2m} - \frac{k_u \cdot k_{c_v}}{(2m)^2} \right\} \tag{3.2}$$

In this equation $2m$ is still the total weight of the graph, k_{u,c_v} is the sum of the weights of the edges from node u to all the nodes of the community of v, k_u is the weighted degree of node u and k_{c_v} is the sum of the degrees of all the nodes in the community of v.

Once the best neighbouring community to assign u to is found, u is inserted into that community and merged to the other nodes. However, if the best gain found is still negative, it means that node u cannot be merged to another community and it remains in its own initial one. Technically, it is possible to merge the nodes after finding the best community, so that nodes u and v become a single one, updating their edge lists coherently. The serial version of Rabbit Order uses this simplification to reduce future computations. At each step, the aggregation is recorded in a dendrogram that will be used later for the ordering generation.

The community detection stops when all the nodes have been assigned to a community and at the end of the cycle the function will have generated a dendrogram. This dendrogram is a data structure composed by multiple trees, identified by top level nodes (nodes that cannot be merged to other) that represent their communities. Each tree contains all the nodes in that community in order of aggregation.

At this point a depth-first search is performed on all the trees in the dendrogram and a new id is assigned to each node in the visiting order. The new ids assigned effectively constitute the sought permutation of the original graph.

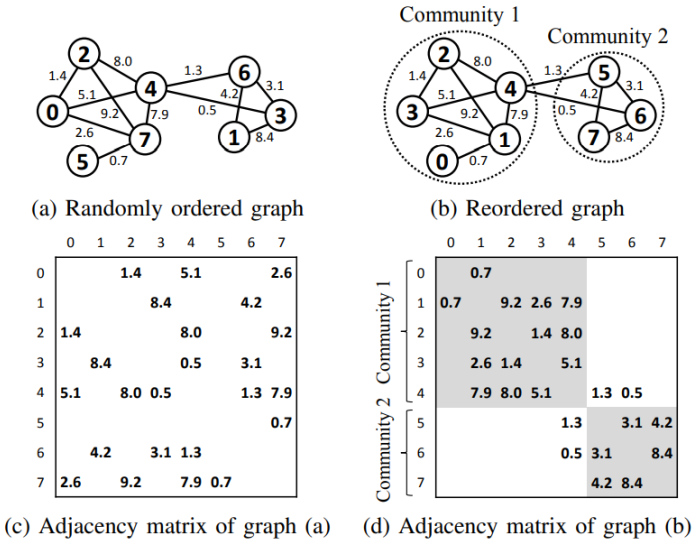

Figure 3.1: Example of weighted graph before and after Rabbit Order (Reproduced from [3])

3.1.2 OMP Parallelization

The authors of Rabbit Order directly provide in the reference paper [3] a parallelized version of the algorithm for shared-memory systems using OpenMP. This type of parallelization is not the main focus of this work, but it could be interesting to discuss the challenges addressed and differences required from the serial implementation in order to better understand the following work.

The main obstacle to overcome when parallelizing an algorithm in shared-memory systems is always dealing with concurrency and data racing. In particular, in the community detection function of Rabbit Order, it is necessary to ensure that, when the node u has to be merged with node v, no other threads are trying to merge their processing node with u. Without using locks to solve this problem, which would have worsen the overall performance, the solution adopted is the so-called lazy aggregation, which consists in registering v community as the target community of u and only perform the proper merge later, when u is processed and inserted in another community. The proper merge will be safely performed by using the atomic compare-and-swap function, invalidating for the other threads the processed node and freeing it at the end of the aggregation. Whenever a thread needs an invalid node, it discards the changes and proceed with another node. Unlike the serial implementation, where the number of operations is deterministically fixed by the sequential processing order, this approach

potentially introduces additional work, leading to a higher overall operation count.

For the ordering generation there are not concurrency problems because each thread processes its own tree of the dendrogram and writes its local ordering that will be later globally merged.

3.2 Target architectures

After analysing the serial and OMP versions of Rabbit Order, we propose a novel distributed implementation designed for distributed-memory systems. This version leverages MPI to enable reordering on large graphs that exceed the memory capacity of a single node. In this overview we will refer to "node" as an individual processing unit, typically a physical machine or logical process with its own private memory and processor. Moreover we define the term "process" as an independent unit of execution with its own local memory space.

3.2.1 Shared vs distributed memory architectures

The OMP parallelization proposed by the authors of Rabbit Order is a suitable solution for shared-memory architectures, where all processors (cores in a node) can simultaneously access the shared memory for inter process communication [35]. The computation is executed by a single process which spawns multiple threads, therefore there's not need to handle the exchange of data through messages but, as read and write operations are executed asynchronously, the work of individual threads can interfere with each other.

In distributed-memory architectures, several nodes are involved, each one with its own local memory. Instead of having a single process with multiple threads working in it, there are multiple processes that cannot access directly to the others' memory. The exchange of data between the nodes is performed through "message passing" over a network interface [35]. Therefore here comes the need not only for the physical connection between the processes, but also for an interface, that is a protocol which all the processes must follow in order to allow correct synchronization and data exchanging between all of them. It's up to the developer and the network designer to define how and when the data has to be transferred.

3.2.2 MPI

The Message Passing Interface (MPI) [32] is a standard specification of message-passing interface for parallel computation in distributed-memory systems [36]. MPI isn't a programming language: as a protocol, it defines some rules to follow, but it also comes with a library of functions for different programming languages, e.g. C, C++ and Fortran, in order to write parallel programs. In the MPI environment, each process in the computation has an identifier called rank (therefore rank will be also used later to refer to processes), its own local memory and it executes independently from others.

The MPI standard offers a series of functions for message passing that can be grouped in three base classes: point-to-point communication, collective communication and one-sided communication. These classes differs for the amount of processes involved in each one [9].

Point-to-point communication is the simplest one, where a process, the sender, sends a message to another process, the receiver, which contains some generic data. MPI provides multiple send and receive modes to support different communication semantics. Blocking operations (e.g., `MPI_Send()`, `MPI_Ssend()`) block the process until the communication can safely proceed or complete. Non-blocking operations (e.g., `MPI_Isend()`, `MPI_Irecv()`) return immediately and require a later completion with `MPI_Wait()`. Standard send (`MPI_Send()`) may or may not wait for the receiver, depending on the system. Synchronous send (`MPI_Ssend()`) waits until the receive is posted. Buffered send (`MPI_Bsend()`) uses user-supplied buffers and does not block. Non-blocking versions exist for all modes and allow overlapping communication and computation. In every scenario, the communication always happens only when the two processes are ready to communicate.

MPI collective communications involve all processes in a communicator and include operations such as broadcast, scatter, gather, and reduction. These operations are blocking by default and are internally optimized, often outperforming equivalent point-to-point implementations. As with point-to-point communication, non-blocking variants (e.g., `MPI_Ibcast()`, `MPI_Ireduce()`) also exist, allowing overlap between communication and computation.

Finally, one-sided communications involve only one process. The key concept of these commu-

nications is the creation of a Remote Memory Access (RMA) window, that is a memory window that is publicly exposed to all the processes in a group (communicator). With the use of `MPI_Put()` and `MPI_Get()` a process (the origin) can, respectively, write or read data from another one (the target). Even though the communication still happens between two processes, this is by concept asynchronous and one-sided, because only the origin process actively exchanges data and performs an action, while the target one passively sends or receives data without any involvement in the communication. The only synchronization required in this type of communication is at the start and at the end of an epoch (which is the execution span occurring between calls to MPI synchronization functions) of data exchanging. The first synchronization required is when creating the RMA window with `MPI_Win_allocate()` or `MPI_Win_create()`, which are collective blocking functions. At the end of all the epochs, the RMA windows are freed with `MPI_Win_free()`. Due to the non-blocking nature of the get and put functions, there are several synchronizations functions to ensure the data readings and writings have been correctly executed. In the proposed algorithm the functions used to ensure synchronization between processes are `MPI_Win_lock_all()` and `MPI_Win_unlock_all()` that basically lock the RMA windows so that data can be safely accessed after acquiring the lock and safely read in the local memory after releasing it. These functions are collective in the sense that all processes must call them in order to ensure the lock, but they are non-blocking: it means that each process calls it independently and without waiting for the others. After the lock all the RMA operations can be executed safely and the results are safe to be read after the unlock.

3.3 Algorithm

In this section we now present our novel implementation of Rabbit Order for distributed-memory systems. The focus of the work is the parallelization of the community detection step of the algorithm, since the ordering generation is trivial once obtained the partition of the initial graph.

As discussed before, the main challenge to address in this parallelization is the synchronization between processes and the exchange of the graph data (e.g. nodes, edges and communities). Although the initial design of the algorithm was guided by the strategy outlined by Sattar et al. in their distributed implementation for the Louvain community detection algorithm [40], key differences and improvements were introduced to address the specific needs of this implementation. Louvain [5] is a serial modularity-based graph community detection algorithm which consists in an incremental aggregation of the nodes in the starting graph based on the modularity gain, just as Rabbit Order does.

Algorithm 2 illustrates the key stages of the process, with a specific focus on community detection. Our implementation involves an iterative, hierarchical, community-based ordering process comprising three main steps. First, the root process P_0 reads the input graph and divides it among the others by assigning a subset of nodes to each based on a particular partitioning function. The second step involves hierarchical community detection: at each iteration, the processes communicate with each other to receive the information about the remote nodes required for computation. They then perform modularity-gain-based community detection on their local nodes. Afterwards, they circulate the results globally and update the dendrogram. The communities found are then reassigned and aggregated for the next iteration. This loop stops when there are no more changes to the community structure of the graph. In the third step, the processes communicate their local reordering to the root process P_0, which merges them into the final ordering. In the next sections every step of the algorithm is described and analysed. Moreover, Table A.1 contains all the notations and variable names used along with their descriptions.

3.3.1 Graph partitioning

The first step of the algorithm consists in the partition of the input data between all the processes. The root process, P_0, reads the graph as an edge list from an input file and, based on a partitioning function, it assigns some nodes along with their adjacency lists to each process. The partitioning function must be chosen ad hoc to evenly distribute the load among all the processes. In our implementation, each process works on a subset of the nodes of the graph and, for this reason, the load to distribute is a set of nodes. The distribution happens in a round-robin way, by simply assigning `v` to `v % size`.

Algorithm 2: Overview of Distributed Rabbit Order using MPI

```
Input  : Graph G = (V, E)
Output: Permutation π : V → ℕ for new nodes ordering
// Root process P_0 reads G as edge list, partitions it and distributes it to other processes
if rank = 0 then
    PartitionGraph(G);
DistributeGraph();
InitialiseGraph();
while improvement do
    foreach process P_i in parallel do
        InitialiseCommunities();
        CollectRemoteNodes();
        DetectCommunities();
        ExchangeCommunities();
        SolveConflicts();
        improvement ← ReassignCommunities();
        if ¬improvement then
            break;
        CollectMissingNodes();
        CoarsenGraph();
reordering ← GenerateReordering();
return reordering;
```

In this way all the processes will have $\lceil \frac{n}{size} \rceil$ nodes to work with. Algorithm 3 describes the steps of PartitionGraph that builds three data structures: map $p2n : P \to \mathcal{P}(V)$ which records the set of nodes for each process, map $p2e : P \to \mathcal{P}(E)$ which records the set of edges for each process and vector $n2p : V \to P$ which records the owner of each node (for instant access later). The vector $n2p$ isn't strictly required for this implementation, due to the fact that the partitioning is deterministic, but in the case of a non-deterministic partitioning (such as a graph-aware one) it is necessary to know the owner of each node with a direct access.

3.3.2 Graph distribution

At this point, the root process knows which nodes are assigned to each process and must physically send them. This step constitutes the first point in the algorithm where message exchange between processes is required. Algorithm 4 describes the process of graph distribution. Note that the syntax used with the get has to be interpreted as `MPI_Get(target process, local buffer, window start, number of values to read)`. In this step root process P_0 must share five different pieces of data to each other process P_i:

- n: the order of the graph (total number of nodes);
- m: the size of the graph (total number of edges);
- $p2n[P_i]$: the assigned nodes for the process P_i;
- $p2e[P_i]$: the assigned edges for the process P_i;
- $n2p$: the partition map.

The maps $p2n$ and $p2e$ must be flattened in order to be correctly shared in a contiguous RMA window. Moreover, other processes must know the starting and the ending position of the data they must read from the flattened maps. For this reason, root process P_0 must also setup two more windows to communicate these pieces of information to each other process P_i. This can be achieved by memorizing

Algorithm 3: Graph Partitioning

```
// Root process P_0 reads G as edge list and partitions it using a partitioning function
Function PartitionGraph(G):
    p2n ← {}; // Initialise empty map p2n : P → P(V)
    p2e ← {}; // Initialise empty map p2e : P → P(E)
    n2p ← {}; // Initialise empty vector n2p : V → P
    foreach node v ∈ V do
        process ← Owner(v);
        p2n[process].push_back(v);
        p2e[process].push_back(all edges whose source is v);
        n2p[v] ← process;
    return (p2n, p2e, n2p);
Function Owner(v):
    return v % size;
```

and sharing the offsets for each process P_i in two arrays $nodesOffsets$ and $edgesOffsets$ such that $nodesOffsets[i]$ tells P_i the position of $p2nFlat$ to start reading from and $nodesOffsets[i+1]$ tells P_i the position of $p2nFlat$ to stop reading from. From here, $nodesOffsets[i+1] - nodesOffsets[i]$ is the number of nodes assigned to P_i. The same happens with $edgesOffsets$ and $p2eFlat$. Note that all the processes will create seven RMA windows, but only root process' ones will be effectively populated and thus valid to be read. At the end of the two access epochs (one to retrieve the size of data to read and one to retrieve the effective data), windows are freed.

3.3.3 Graph initialisation

Now that all processes received their own data, they can start building the local graph on which they'll do the computation. Hence, in this step, every process will read its edge list and build the local graph as an adjacency list, that is populating a map $neighbours : localNodes \to \mathcal{P}(V)$. The construction of this map is convenient for later operations, such as getting the degree of a node or its neighbours, without scrolling the entire edge list.

3.3.4 Communities initialisation

Once all the preliminary steps have been completed and the initial data has been distributed, the proper community detection iteration can begin. Starting from the local graph, each process initialises the data structures required for community detection. These are: $n2c : V \to V$ which records the community assigned to an arbitrary node; $tot : V \to \mathbb{N}$ which records the total weight of a community, i.e. the sum of the weighted degrees of all its nodes (since we are assuming an unweighted graph, integer numbers suffice); and $in : V \to \mathbb{N}$ which records the internal weight of a community, i.e. the sum of all the edges between nodes inside the community. Algorithm 5 describes this step. At this stage, each node is assigned to its own community.

3.3.5 Remote nodes collection

Before the community detection computation can be performed, each process must be aware of all the information relating to the so-called remote or ghost nodes. These are the nodes owned by other processes to which local nodes are connected by some edges.

In this step each process must first scan through all of the neighbours of its nodes to detect the remote ones, after which a message exchange phase will begin. In this phase, every process shares its whole (flattened) adjacency list ($neighbours$) with all the others. Algorithm 6 describes this process. It is interesting to note that in the Louvain reference paper [40] all the communications between processes were point-to-point and blocking. In this implementation, however, all the inter-process communication is realised using RMA functions, meaning that each process works asynchronously and it is not blocked by any of the others once the required data has been obtained.

Algorithm 4: Graph Distribution

```
// All processes create RMA windows, but only P_0 populates them. All processes read their
   data from P_0
1  Function DistributeGraph():
       // Initialise local graph data
2      nodesOffsetStart, nodesOffsetEnd, edgesOffsetStart, edgesOffsetEnd ← 0;
3      localNodes, localEdges ← {};
4      if rank = 0 then
5          p2nFlat, nodesOffsets ← Flatten(p2n);
6          p2eFlat, edgesOffsets ← Flatten(p2e);
       // Create the seven required windows
7      MPI_Win_create(nWin, mWin, nodesOffsetsWin, edgesOffsetsWin);
8      MPI_Win_create(p2nFlatWin, p2eFlatWin, n2pWin);
       // Beginning of the FIRST access epoch
9      MPI_Win_lock_all(nWin, mWin, nodesOffsetsWin, edgesOffsetsWin);
10     if rank ≠ 0 then
11         MPI_Get(0, n, nWin, 1);
12         MPI_Get(0, m, mWin, 1);
13         MPI_Get(0, nodesOffsetStart, nodesOffsetsWin + rank, 1);
14         MPI_Get(0, nodesOffsetEnd, nodesOffsetsWin + rank + 1, 1);
15         MPI_Get(0, edgesOffsetStart, edgesOffsetsWin + rank, 1);
16         MPI_Get(0, edgesOffsetEnd, edgesOffsetsWin + rank + 1, 1);
17     MPI_Win_unlock_all(nWin, mWin, nodesOffsetsWin, edgesOffsetsWin);
       // End of the FIRST access epoch
       // Beginning of the SECOND access epoch
18     MPI_Win_lock_all(p2nFlatWin, p2eFlatWin, n2pWin);
19     if rank ≠ 0 then
20         ln ← nodesOffsetEnd − nodesOffsetStart; // Number of assigned nodes
21         le ← edgesOffsetEnd − edgesOffsetStart; // Number of assigned edges
22         MPI_Get(0, localNodes, p2nFlatWin + nodesOffsetStart, ln);
23         MPI_Get(0, localEdges, p2eFlatWin + edgesOffsetStart, le);
24         MPI_Get(0, n2p, n2pWin, n);
25     else
26         localNodes ← p2n[rank];
27         localEdges ← p2e[rank];
28     MPI_Win_unlock_all(nWin, mWin, nodesOffsetsWin, edgesOffsetsWin);
       // End of the SECOND access epoch
       // Free the windows
29     MPI_Win_free(nWin, mWin, nodesOffsetsWin, edgesOffsetsWin);
30     MPI_Win_free(p2nFlatWin, p2eFlatWin, n2pWin);
31 Function Flatten(map):
32     mapFlat ← {}; // Initialise empty vector for data
33     offsets ← {}; // Initialise empty vector for offsets
34     offsets.push_back(0);
35     foreach vector v ∈ map do
36         mapFlat.insert(v);
37         offsets.push_back(mapFlat.size());
38     return (mapFlat, offsets);
```

Algorithm 5: Communities initialisation

```
// Initialise the local community structure
Function InitialiseCommunities():
    foreach node v ∈ localNodes do
        Insert(v, v, 0);

Function Insert(node, community, n2cWeight):
    n2c[node] ← community;
    tot[node] ← tot[node] + deg(node);
    in[node] ← in[node] + 2 * n2cWeight + self_loops(node);
```

Algorithm 6: Remote nodes collection

```
// Collect remote nodes from other processes
Function CollectRemoteNodes():
    getList ← {}; // Initialise empty vector for nodes to retrieve
    foreach node v ∈ localNodes do
        foreach node u ∈ neighbours[v] do
            if Owner(u) ≠ rank and ¬getList.contains(u) then
                getList.push_back(u);

    // Create the windows
    neighboursFlat, offsets ← Flatten(neighbours);
    MPI_Win_create(offsetsWin, neighboursFlatWin);
    getOffsets ← {};
    MPI_Win_lock_all(offsetsWin);
    foreach node v ∈ getList do
        start, end ← 0;
        MPI_Get(Owner(v), start, offsetsWin + v, 1);
        MPI_Get(Owner(v), end, offsetsWin + v + 1, 1);
        getOffsets.push_back(start);
        getOffsets.push_back(end);
    MPI_Win_unlock_all(offsetsWin);
    remoteNodes ← {}; // Initialise empty vector for collected remote nodes
    i ← 0; // Initialise iterator for offsets
    MPI_Win_lock_all(neighboursFlatWin);
    foreach node v ∈ getList do
        ln ← offsets[2 × i + 1] − offsets[2 × i]; // Number of assigned nodes
        MPI_Get(Owner(v), neighbours, neighboursFlatWin + offsets[2 × i], ln);
        remoteNodes.push_back(v);
        i++;
    MPI_Win_unlock_all(neighboursFlatWin);
    MPI_Win_free(offsetsWin, neighboursFlatWin);
    // Update local community structure
    foreach node v ∈ remoteNodes do
        Insert(v, v, 0);
```

3.3.6 Communities detection

In this step each process works on its local graph in order to assign a community for each of the *localNodes*. The procedure is the same as it happens with Rabbit Order or Louvain:

$$n2c[v] \leftarrow n2c[u^*], \quad \text{where } u^* = \arg \max_{u \in \text{neighbours}[v]} \Delta Q(v,u) > 0 \tag{3.3}$$

The modularity gain is computed using a heuristic based on Equation 3.2, as the focus is on the resulting value rather than the exact mathematical expression:

$$\Delta Q(v,u) = k_{v,c_u} - \frac{k_v \cdot k_{c_u}}{2m} \tag{3.4}$$

Algorithm 7 describes this step. The only notable difference from the original Rabbit Order is that, in this implementation, nodes are simply assigned to a community and there is no aggregation yet. This is because nodes can also be assigned to a community represented by remote nodes, and aggregation would be difficult to perform at the moment (requires communication). Therefore, this will be solved later in a centralised manner.

Algorithm 7: Community detection

```
   // Assign a community to each node
 1 dendrogram ← CommunityDetection();
 2 Function DetectCommunities():
       // Perform incremental aggregation
 3     foreach v ∈ localNodes in random order do
 4         u ← neighbour of v that maximizes ΔQ(v, u);
 5         if ΔQ(v, u) > 0 then
 6             Remove(v, n2c[v], weight to n2c[v]);
 7             Insert(v, n2c[u], weight to n2c[u]);

 8 Function Remove(node, community, n2cWeight):
 9     n2c[node] ← −1; // Invalidate community
10     tot[node] ← tot[node] − deg(node);
11     in[node] ← in[node] − 2 ∗ n2cWeight − self_loops(node);
```

3.3.7 Communities exchange

After proper community detection, all processes must communicate to update each other on the communities found and to aggregate the graph in preparation for the next iteration. This preliminary step therefore involves exchanging community assignments, first by sharing the size and values of *localNodes* and their respective *n2c*, and then by reading these data from the other processes. This ensures that the update is circulated globally, as all processes must be aware of the new community structure for future computations and updates. Algorithm 8 describes the steps of this process.

3.3.8 Conflicts solution

As described in the distributed Louvain reference paper [40], when community detection is performed without aggregation, nodes can form chains where node 1 is assigned to node 2, node 2 to node 3, and so on. These chains can also be closed, such as 1, 2, 3, 1. This is not usually a problem, but it can lead to misunderstandings when computing unique communities. In the reference paper, the authors solved these conflicts locally by changing the values of lower communities to higher ones, and by using point-to-point communication to identify the required node's community. Fortunately, we have already circulated the community structure globally [subsection 3.3.7], so a path compression algorithm is all that is needed to solve the conflicts. This implementation uses a variant of the Union-Find algorithm [42], also known as Disjoint Set Union. Specifically, path compression is applied, meaning each node points directly to the root of its community.

Algorithm 8: Communities exchange

```
   // Exchange the updated community structure globally
 1 Function ExchangeCommunities():
 2     count ← localNodes.size();
 3     foreach node v ∈ localNodes do
 4         communities.push_back(n2c[v]);
       // Create the windows
 5     MPI_Win_create(countWin, nodesWin, communitiesWin);
 6     getCounts ← {}; // Initialise empty array for received counts
 7     MPI_Win_lock_all(countWin);
 8     foreach process p ∈ P do
 9         if p ≠ rank then
10             MPI_Get(p, getCounts[p], countWin, 1);
11     MPI_Win_unlock_all(countWin);
12     getNodes ← {}; // Initialise empty array for received nodes
13     getCommunities ← {}; // Initialise empty array for received communities
14     MPI_Win_lock_all(nodesWin, communitiesWin);
15     foreach process p ∈ P do
16         if p ≠ rank then
17             MPI_Get(p, getNodes.end(), nodesWin, getCounts[p]);
18             MPI_Get(p, getCommunities.end(), communitiesWin, getCounts[p]);
19     MPI_Win_unlock_all(nodesWin, communitiesWin);
20     MPI_Win_free(countWin, nodesWin, communitiesWin);
       // Update local community mapping
21     for i ← 1 to getNodes.size() do
22         n2c[getNodes[i]] ← getCommunities[i];
```

3.3.9 Communities reassignment

After applying path compression to the community structure, it is possible to gain an understanding of the unique communities identified by creating a map, $c2n : V \to \mathcal{P}(V)$, which records all the nodes associated with a community. Furthermore, the found community structure must be recorded in the final *dendrogram* required for reordering. This step is crucial for two reasons: firstly, to determine whether the algorithm should proceed, and secondly, to reassign communities to processes if necessary.

If the number of unique communities found in the previous iteration of the algorithm is the same as the one obtained at this point, it means that it is no longer possible to improve the modularity of the graph; therefore, the optimal community structure has been found.

If this does not happen, after aggregating the graph, the algorithm will require the aggregated nodes to be partitioned again to ensure load balancing and prevent processes from becoming idle. For this reason, the unique communities will be renumbered and then reassigned to all processes in this step. Note that this reassignment is distributed: all processes use the same partitioning function, eliminating the need to read the partition from the root process and reducing communication. Algorithm 9 describes this step.

3.3.10 Missing nodes collection

Before aggregation, it is necessary to ensure that each process has all the data for the nodes belonging to the assigned communities. This step requires communication and is almost identical to CollectRemoteNodes(): *getList* is built by checking whether each node in $c2n$ of the assigned communities is present in *localNodes* or *remoteNodes*. If it has been collected previously, it is not inserted in *getList*. Otherwise, it is inserted and collected with a RMA communication.

Algorithm 9: Communities reassignment

```
   // Renumber communities and reassign for next iteration
1  Function ReassignCommunities():
2      c2n ← {}; // Initialise empty map c2n : V → P(V)
3      foreach node v ∈ V do
4          c2n[Owner(v)].push_back(v);
5      if prevCommunities = c2n.size() then
6          return false;
7      prevCommunities ← c2n.size();
8      Add found communities to dendrogram;
9      Renumber found communities;
10     Update n2p;
11     return true;
```

3.3.11 Graph Coarsening

The final step in the iteration process involves aggregating all nodes belonging to the same community to create a coarse graph. This is a graph in which the nodes are the communities of the original graph, and the source and destination of the edges are the communities of the original source and destination nodes. Furthermore, all edges with the same new source and destination will be combined into a single edge with the correct accumulated weight, thereby reducing future computation.

Algorithm 10 describes the steps of this process. In the Louvain reference paper [40], the aggregation and graph distribution are always performed centrally by the root process P_0. This creates a bottleneck around its computation, rendering all other processes idle. In this implementation, however, each process coarsens its own new nodes, meaning that in future iterations, the graph will already be partitioned and distributed, and the only communication required will be for collecting remote nodes.

Algorithm 10: Graph coarsening

```
   // Aggregate local nodes into the new graph
1  Function CoarsenGraph():
2      localNodes ← {}; // Empty localNodes for new graph
3      foreach community c ∈ c2n do
4          if Owner(c) ≠ rank then
5              continue;
6          localNodes.push_back(c);
           // Compute new nodes and adjacency list
7          nc ← {}; // Initialise empty array for neighbouring communities
8          nw ← {}; // Initialise empty array for neighbouring weights
9          foreach node v ∈ c2n[c] do
10             foreach neighbour u ∈ neighbours[n] do
11                 nc.insert(n2c[u]);
12                 nw[n2c[u]] ← nw[n2c[u]] + weights[v][u];
           // Insert data in new coarse graph
13         foreach neighbour community d ∈ nc do
14             neighbours[c].push_back(d);
15             weights[c].push_back(nw[d]);
```

3.3.12 Reordering generation

Reordering generation is executed in the same way as the shared-memory implementation of Rabbit Order. That is to say, each process generates its local reordering by visiting the communities assigned to it in the last iteration of the dendrogram. Then, it writes the final reordering directly to the *reordering* array of the root process P_0 using `MPI_Put()`. To prevent concurrency problems in the memory of P_0, each process must know where to write its data. For this reason, a preliminary step is necessary where all processes share their *localReordering* size, and each process reads the previous ones with `MPI_Get()` to compute its $offset$. Algorithm 11 describes the steps of this process. Note that, as before, all the processes create the window for the final reordering, but only the root process's window will be valid at the end of the function.

Algorithm 11: Reordering generation

```
   // Generate the new reordering
1  Function GenerateReordering():
2      localReordering ← {}; // Initialise empty array for local reordering
3      reordering ← {}; // Initialise empty array for final reordering
4      foreach community c ∈ c2n do
5          if Owner(c) ≠ rank then
6              continue;
7          foreach node v ∈ dendrogram[c] in DFS visiting order do
8              localReordering.push_back(v);
9          localSize ← localReordering.size();
10         sizes ← {}; // Initialise empty array for other processes' sizes
           // Create the windows
11         MPI_Win_create(localSizeWin, reorderingWin);
           // Read other's localSize
12         MPI_Win_lock_all(localSizeWin);
13         for i ← 0 to rank − 1 do
14             MPI_Get(i, sizes[i], localSizeWin, 1);
15         MPI_Win_unlock_all(localSizeWin);
16         offset ← 0;
17         for i ← 0 to rank − 1 do
18             offset ← offset + sizes[i];
           // Write final reordering
19         MPI_Win_lock_all(reorderingWin);
20         if rank ≠ 0 then
21             MPI_Put(0, localReordering, reorderingWin + offset, localSize);
22         else
23             reordering.insert(localReordering);
24         MPI_Win_unlock_all(reorderingWin);
25         MPI_Win_free(localSizeWin, localReorderingWin);
26         return reordering;
```

3.3.13 Correctness

To verify the correctness of the algorithm, its output is compared with the results obtained using the serial or shared-memory implementations of the Rabbit Order algorithm or the Louvain algorithm. In particular, it is possible to compare the number of unique communities identified by the algorithm and the graph's modularity after the community detection. Due to the parallelisation, results obtained

from the same input data will not be identical. However, if the computations are performed correctly, the number of communities and modularity are comparable quantitative indicators that directly reflect the effect of the algorithm on the graph.

3.3.14 Complexity

The time complexity of the algorithm can be analysed by evaluating the complexity of each step. For communication processes, the general time complexity of a single `MPI_Get()` can be approximated as $\mathcal{O}(\alpha+\beta \cdot n)$, where α is the latency overhead (communication setup time), β is the inverse throughput (time to send a byte of data or, more generically, an element) and n is the number of transferred bytes. If the latency overhead is very low (e.g. when using InfiniBand), then α is negligible and the complexity depends directly on the amount of transferred data. Assuming this, and letting n_i and m_i represent the nodes and edges assigned to each process respectively, the complexity of each step for a single iteration and a single process of the algorithm will be as follows:

1. PartitionGraph: reads the whole edge list and, for each edge, it assigns the source node (if not already done) and the edge to relative process. Therefore it will have a complexity of $\mathcal{O}(m)$;
2. DistributeGraph: P_0 calls Flatten twice, with a total complexity of $\mathcal{O}(m+n)$. Then, every process reads from P_0 six single values plus the three vectors. Hence, the total complexity will be $\mathcal{O}(m+n+\beta(n \cdot (size+1)+m))$;
3. InitialiseGraph: each process reads the obtained edge list. Therefore, the complexity is $\mathcal{O}(m_i)$;
4. InitialiseCommunities: each process initialises the community structure for its n_i nodes. Hence, the complexity is $\mathcal{O}(n_i)$;
5. CollectRemoteNodes: each process builds the *getList* with a complexity of $\mathcal{O}(m_i)$. Let $r_i = |getList|$ the number of remote nodes to collect, then each process reads a total of $2 \cdot r_i + \sum_{v \in getList} deg(v)$ values from the others. Finally, it will update the local structure with r_i operations. Therefore, the complexity is $\mathcal{O}(m_i + \beta(2 \cdot r_i + \sum_{v \in getList} deg(v)) + r_i)$;
6. DetectCommunities: each process reads the adjacency list of each node to find the best community. Hence, the complexity is $\mathcal{O}(m_i)$;
7. ExchangeCommunities: each process builds the *communities* list with a complexity of $\mathcal{O}(n_i)$. Then, in the communication, each process reads the count, the nodes and the communities from each other, for a total of $(size-1)+2 \cdot (n-n_i)$ values. At the end, each process updates the local community structure with a complexity of $\mathcal{O}(n-n_i)$. Hence, the total complexity is $\mathcal{O}(\beta(size-1+2 \cdot (n-n_i))+n)$;
8. SolveConflicts: each process executes the union-find with path compression with a complexity of $\mathcal{O}(n \cdot \alpha(n)) \approx \mathcal{O}(n)$;
9. ReassignCommunities: each process renumbers the communities with a complexity of $\mathcal{O}(n)$;
10. CollectMissingNodes: exactly as CollectRemoteNodes, let $r_i = |getList|$, then the complexity is $\mathcal{O}(m_i + \beta(2 \cdot r_i + \sum_{v \in getList} deg(v)) + r_i)$;
11. CoarsenGraph: each process coarsens the graph by finding its own communities, aggregating the nodes and saving the new graph for a final complexity of $\mathcal{O}(n+m_i+\frac{n}{size})$;
12. CoarsenGraph: let n_l be the number of the local nodes to reorder, each process generates its local reordering with a complexity of $\mathcal{O}(n_l)$, then reads from other processes a maximum of $size$ values and writes to root process its reordering. Hence, the final complexity is $\mathcal{O}(n_l+\beta(size+n_l))$;

The complexity of input data partitioning is obtained by adding together the complexities of PartitionGraph, DistributeGraph and InitialiseGraph. The complexity of community detection

is then obtained by adding together the complexities of all the iteration steps and multiplying by the number of iterations I. Assuming that $n, m \gg size$, then:

$$\mathcal{O}_{Partitioning} = \mathcal{O}(m + m + n + \beta(n \cdot (size + 1) + m) + m_i) = \mathcal{O}(m + n + \beta(n \cdot size + m)) \tag{3.5}$$

$$\mathcal{O}_{CommunityDetection} = \mathcal{O}(I \cdot (n + m_i + \beta(n + \sum_{v \in getList} deg(v)))) \tag{3.6}$$

$$\mathcal{O}_{ReorderingGeneration} = \mathcal{O}(n_l + \beta(size + n_l)) = \mathcal{O}(n_l + \beta(n_l)) \tag{3.7}$$

The obtained complexities are all of the form $A + \beta B$, where A is the computational complexity and B is the communication complexity, which depends on the bandwidth β. From Equation 3.5, it can be seen that for dense graphs ($m \gg n$), the complexity is linear in m. For sparse graphs ($m \approx n$), however, the communication complexity increases with the number of processes.

Furthermore, Equation 3.6 shows that, in addition to depending on the number of iterations, the complexity of both computations and communication depends heavily on how the graph, and especially the edges, are partitioned among the processes. Note that the number of iterations strictly depends on the graph's structure: in the worst-case scenario, it can be n, meaning that only one node is aggregated in each iteration. In real-world cases, however, I is generally much lower than n.

Finally, Equation 3.7 shows that the complexity of reordering generation depends heavily on the number of nodes (n_l) read by each process in its assigned communities. This cannot easily be determined in advance when only the top levels of the dendrogram are partitioned because it depends on how the communities were previously generated.

In the worst-case scenario, the complexity of Distributed Rabbit Order is shown in Equation 3.8, which occurs when the input graph is densely connected and only one node is aggregated at each iteration, with all nodes in the graph required for local computations.

$$\mathcal{O}_{DistributedRabbitOrder}(m + n(n + m + \beta(n + m)) + \beta(n \cdot size + m)) \tag{3.8}$$

4 Experimental setup

4.1 Dataset

Table 4.1 shows the graphs that were used to test the algorithm. The networks were chosen from real-world datasets from SNAP [29]. Our experimentation has covered different domains of networks, including social networks, the internet, peer-to-peer networks, road networks, networks with ground-truth communities and Wikipedia networks. These networks all exhibit different structural and organisational properties. This also enables us to evaluate the performance of our algorithms with worst-case inputs. The graphs used in our experiments range in size from several hundred thousand to millions of vertices and edges. Furthermore, this enables us to compare the results obtained with those in the Louvain reference paper [40].

4.2 Environment

The algorithm has been implemented in C++ using the MPI 3.1 standard [33] and then compiled using `GCC-9.1.0` [14] and `MPICH-3.2.1` [43]. The tests were executed on the UniTrento HPC cluster [1] in the configuration shown in Table 4.2. The code was compiled using the `-O3` compilation flag and executed with the `mpirun` launcher, with varying numbers of processes ranging from 1 to 64. Execution times were measured using `MPI_Wtime()` to ensure consistent time measurements across processes. Each test was executed multiple times (typically five runs), and the average execution time

was used to reduce the influence of external noise and variability in runtime measurements.

Table 4.1: Graph used in the experiments

Graph	Nodes	Edges	Density (%)	Description
Email-Eu-core [46]	1005	25,571	2.53171	Email network from a European research inst.
Ego-Facebook [31]	4039	88,234	0.54086	Social circles ('friends lists') from Facebook
Wiki-Vote [28]	7115	103,689	0.20482	Wikipedia who-votes-on-whom network
p2p-Gnutella08 [22]	6301	20,777	0.05233	Snapshot of Gnutella network
p2p-Gnutella09 [23]	8114	26,013	0.03951	peer-to-peer file sharing network
p2p-Gnutella04 [21]	10,876	39,994	0.03381	different dates of August 2002
p2p-Gnutella25 [24]	22,687	54,705	0.01063	
p2p-Gnutella30 [25]	36,682	88,328	0.00656	
p2p-Gnutella31 [26]	62,586	147,892	0.00378	
Soc-Slashdot0902 [27]	82,168	948,464	0.01405	Slashdot social network from February 2009

Table 4.2: Execution environment

Properties	Parameters
Architecture:	x86_64
Model name:	Intel(R) Xeon(R) Gold 6140M
CPU MHz:	2300.000
CPUs:	72
Threads per core:	1
Cores per socket:	18
Sockets:	4
NUMA nodes:	4
L1d cache:	32K
L1i cache:	32K
L2 cache:	1024K
L3 cache:	25344K

5 Results

The results of the tests were collected and analysed in order to evaluate the strong scaling (speedup) and the efficiency of the implementation of the proposed algorithm. Additionally, the execution times of individual steps have been collected in order to facilitate the visualisation and identification of steps that require more time.

5.1 Strong scaling and efficiency

In order to estimate the performance of the parallelisation of a generic algorithm, it is possible to compute the strong scaling (also known as speedup) of the application. This is achieved by fixing the size of the input data and measuring the time it takes while increasing the number of parallel computational units [17]. In this case, processes are measured. The estimation of strong scaling can be facilitated through the utilisation of the following Equation 5.1:

$$Speedup = \frac{t(1)}{t(N)} \tag{5.1}$$

In this context, the term $t(1)$ is used to denote the computational time required for the execution of software with a single process, whereas $t(N)$ indicates the computational time required for the execution of the same software with N processes. In an ideal scenario, the software would exhibit

a linear speedup that is equal to the number of processes ($speedup = N$), thereby ensuring that each process contributes with 100% of its computational power. It is regrettable that this is a highly ambitious objective for real-world applications to accomplish, particularly in the context of distributed-memory parallelisations, where the time required for data exchange must be considered.

In order to provide a clearer and more intuitive assessment of performance, speedup is often converted into efficiency, which expresses how close the measured speedup is to the ideal linear speedup. Efficiency is defined as the ratio between the speedup and the number of processes used, multiplied by 100 to obtain a percentage (Equation 5.2):

$$Efficiency = \frac{Speedup}{N} \cdot 100 = \frac{t(1)}{N \cdot t(N)} \cdot 100 \tag{5.2}$$

The speedup and efficiency of each input graph have been plotted in Figure 5.1 and Figure 5.2. In the context of smaller networks, the parallelisation process did not result in a significant enhancement of performance. This is particularly evident when an increased number of processes are employed, as the communication overhead and the limited number of nodes per process contribute to a substantial reduction in efficiency. But, when increasing the size of the input graph, speedup and efficiency register a sensible improvement.

Moreover, the tests show that the application becomes more efficient with a decreasing graph density. In fact, tests on p2p-Gnutella31 demonstrate a higher speedup than tests on Slashdot0902, despite the latter being a larger network in terms of both order and size. This behaviour can only be explained by comparing the densities of the two graphs: a sparser graph has a lower average neighbour count per node than a denser graph. Therefore, the step where information from remote nodes is collected requires less communication and data transfer between processes, resulting in improved overall performance due to minimised communication.

In most tests, the efficiency measured exceeds 100%, indicating a case of superlinear speedup. Despite its counter-intuitive nature, this behaviour has been thoroughly documented and may arise as a consequence of several hardware- and algorithm-related factors. One of the most common explanations for this phenomenon pertains to the memory hierarchy. When a problem is distributed across multiple processes, the working set for each process becomes smaller and may fit entirely within faster cache levels (such as L2 or L3). This results in significantly reduced cache misses compared to sequential execution.

As asserted by Ristov et al. in their seminal work on superlinear speedups [38], such phenomena are predominantly catalysed by augmented effective cache utilisation. A detailed taxonomy is provided by them, identifying multiple causes, including increased total cache capacity, enhanced data locality, and the exploitation of shared cache across cores. In particular, loosely coupled parallel applications stand to benefit from private L2 caches and a shared L3 cache, which, when employed in combination, can result in enhanced access times and elevated throughput. Additional sources of superlinear speedup include superior memory access patterns on NUMA architectures. The study also emphasises that superlinear speedup typically occurs only within specific ranges of problem size and process count. Beyond these limits, performance gains tend to stabilise or degrade due to communication overheads or memory contention, as it is observed when increasing the number of processes.

In conclusion, the observation of efficiency values in excess of 100% does not signify measurement error. Rather, it is indicative of the manner in which parallel execution is capable of leveraging the underlying hardware with greater efficiency in comparison to the sequential baseline. This is particularly evident in the domains of memory access and cache utilisation.

5.2 Execution times

During testing, execution times for each step of the algorithm were collected to demonstrate dependency on order or graph size and identify bottlenecks. By plotting all the execution times against the order of the graph for a fixed number of processes (Figure 5.3, Figure 5.5, Figure 5.7), it is possible to distinguish between execution times that scale linearly with increasing order and those that show oscillations instead. The same occurs when plotting execution times against graph size.

The tests show that partitioning time is the only factor that scales linearly with an increase in

graph size. The other timescales increase regularly as the order of the graph increases. Of all the timescales, that required for collecting remote nodes is always the greatest. However, increasing the number of processes reduces this time proportionally. With more processes, the graph nodes are distributed more evenly among them, but the processes need to collect fewer nodes on average. This reduces the amount of data read during communication, which explains why the overall times decrease.

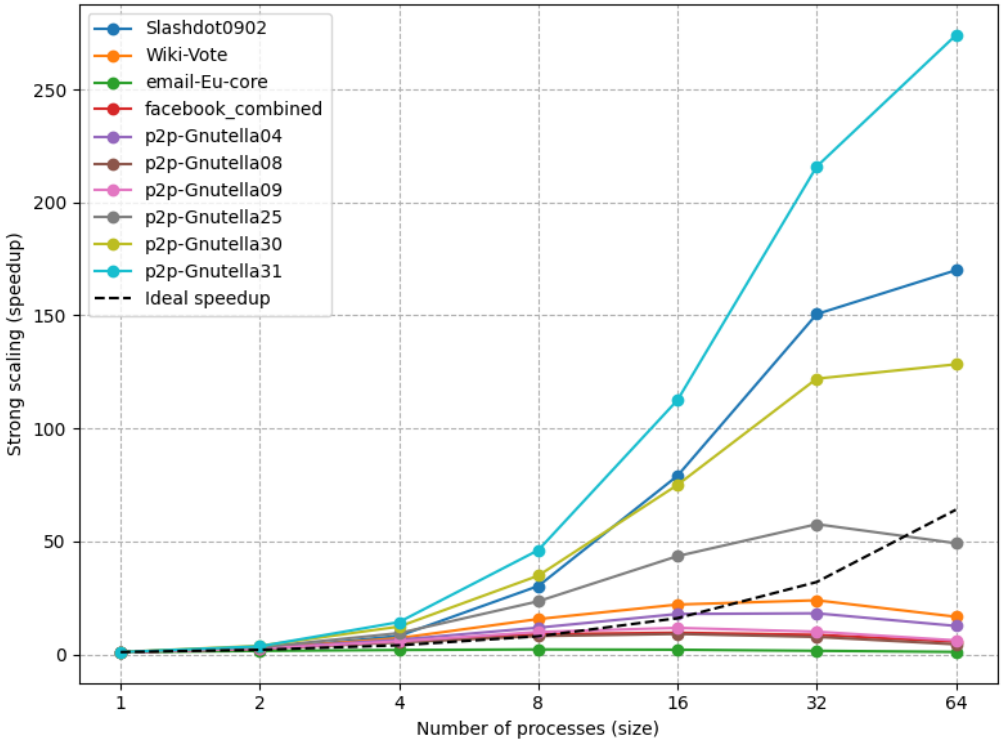

Figure 5.1: Strong Scaling for all the dataset's graphs

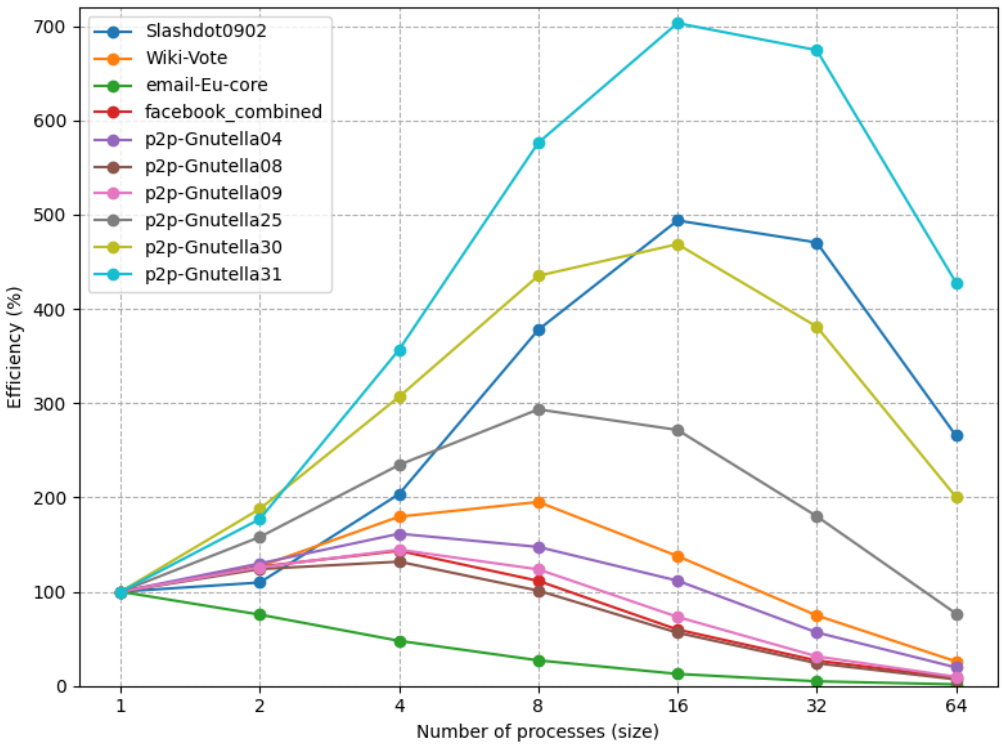

Figure 5.2: Efficiency for all the dataset's graphs

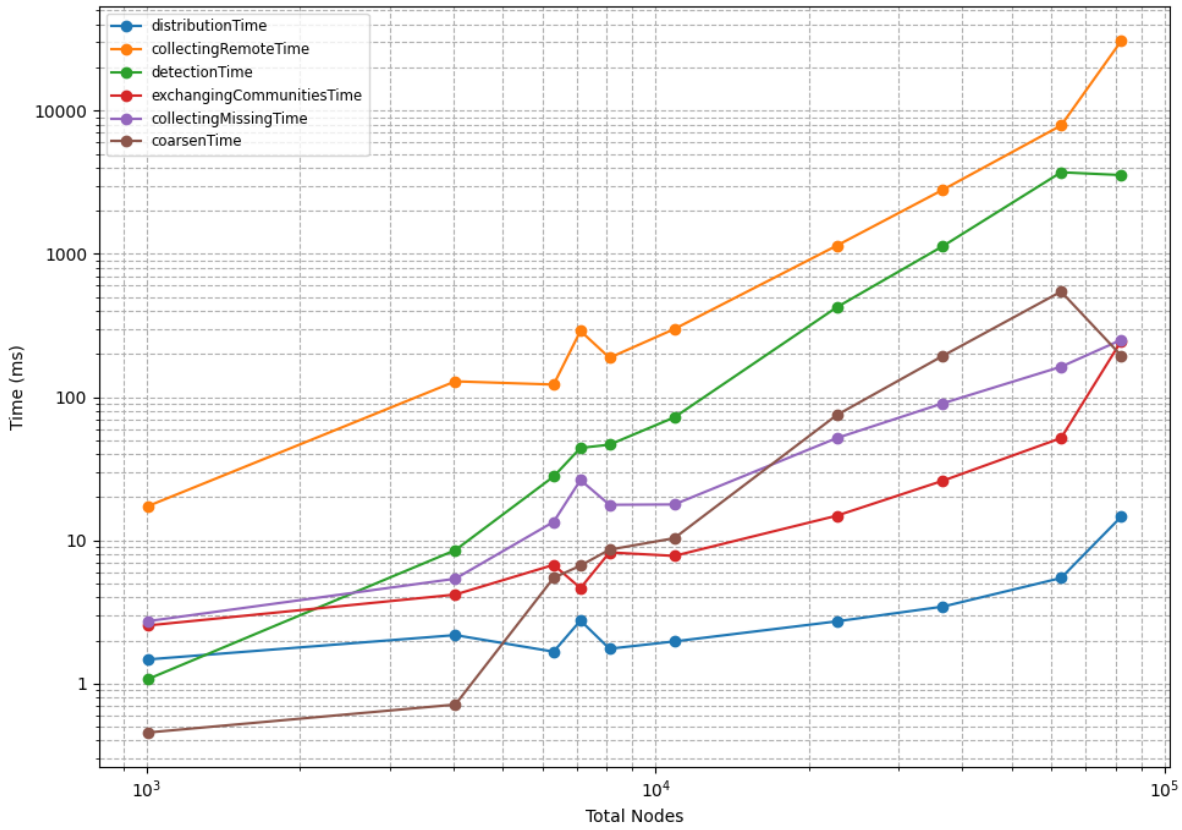

Figure 5.3: Time Breakdown against Total Nodes for all the dataset's graph and fixed size = 4

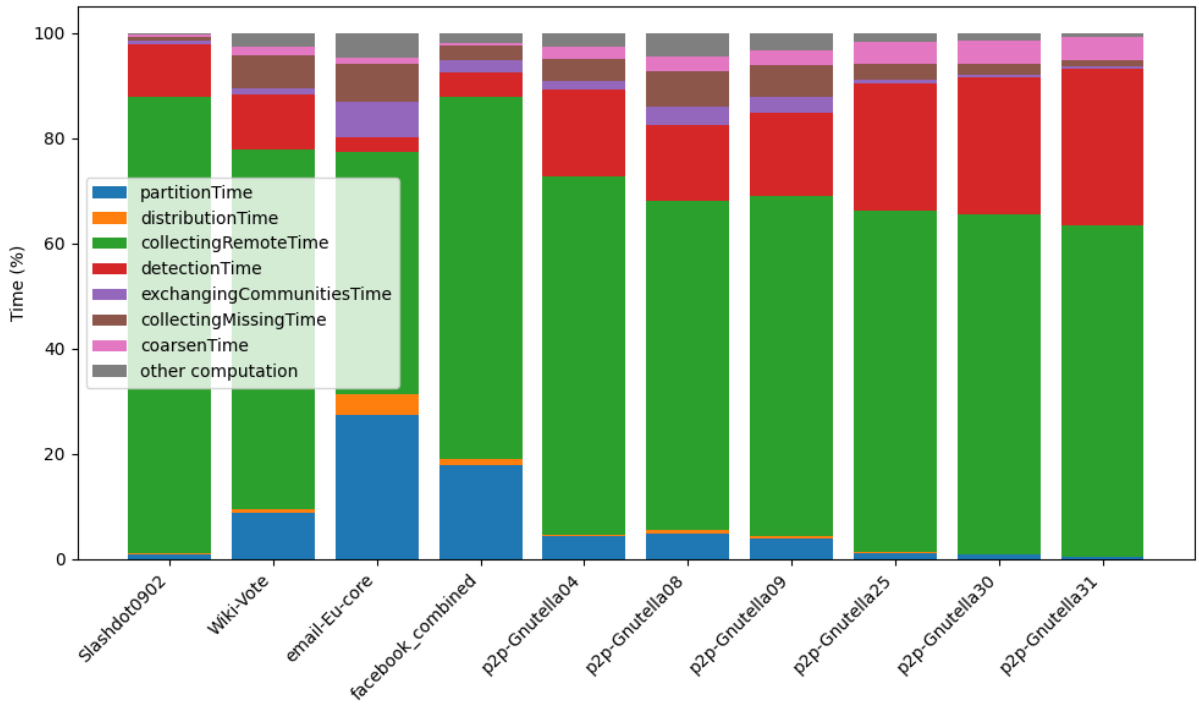

Figure 5.4: Time Breakdown by percentage for all the dataset's graph and fixed size = 4

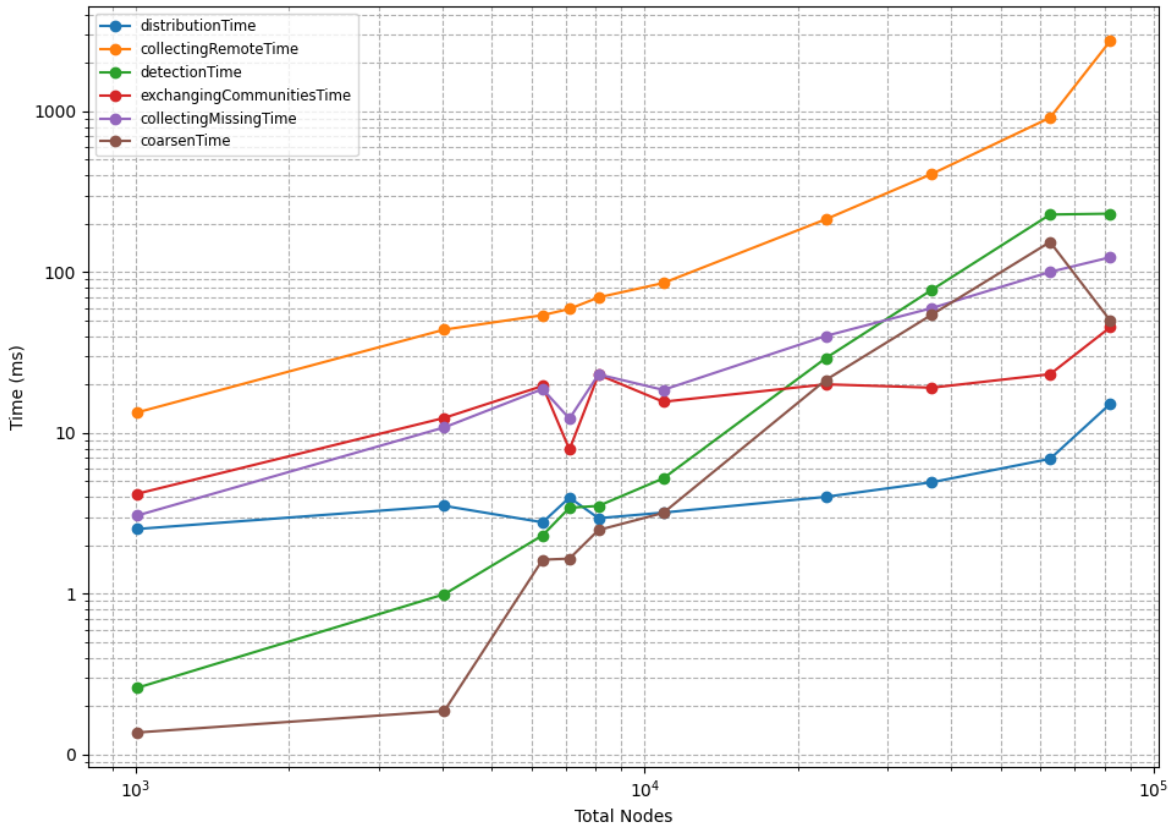

Figure 5.5: Time Breakdown against Total Nodes for all the dataset's graph and fixed size = 16

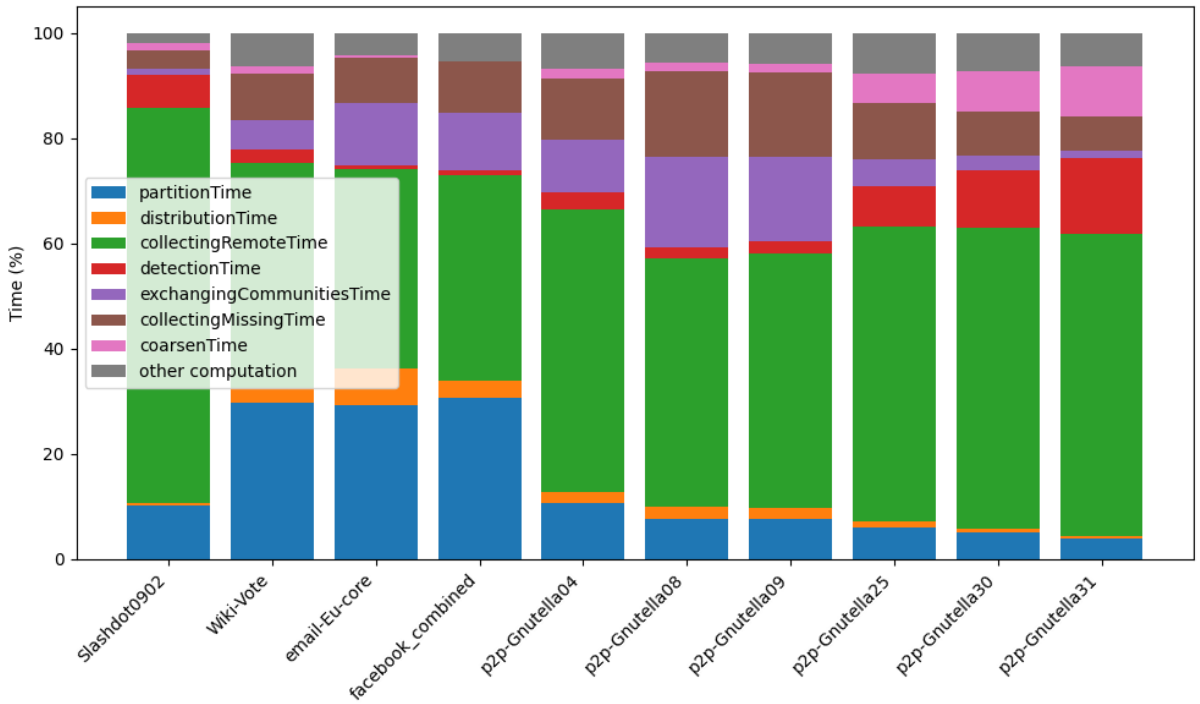

Figure 5.6: Time Breakdown by percentage for all the dataset's graph and fixed size = 16

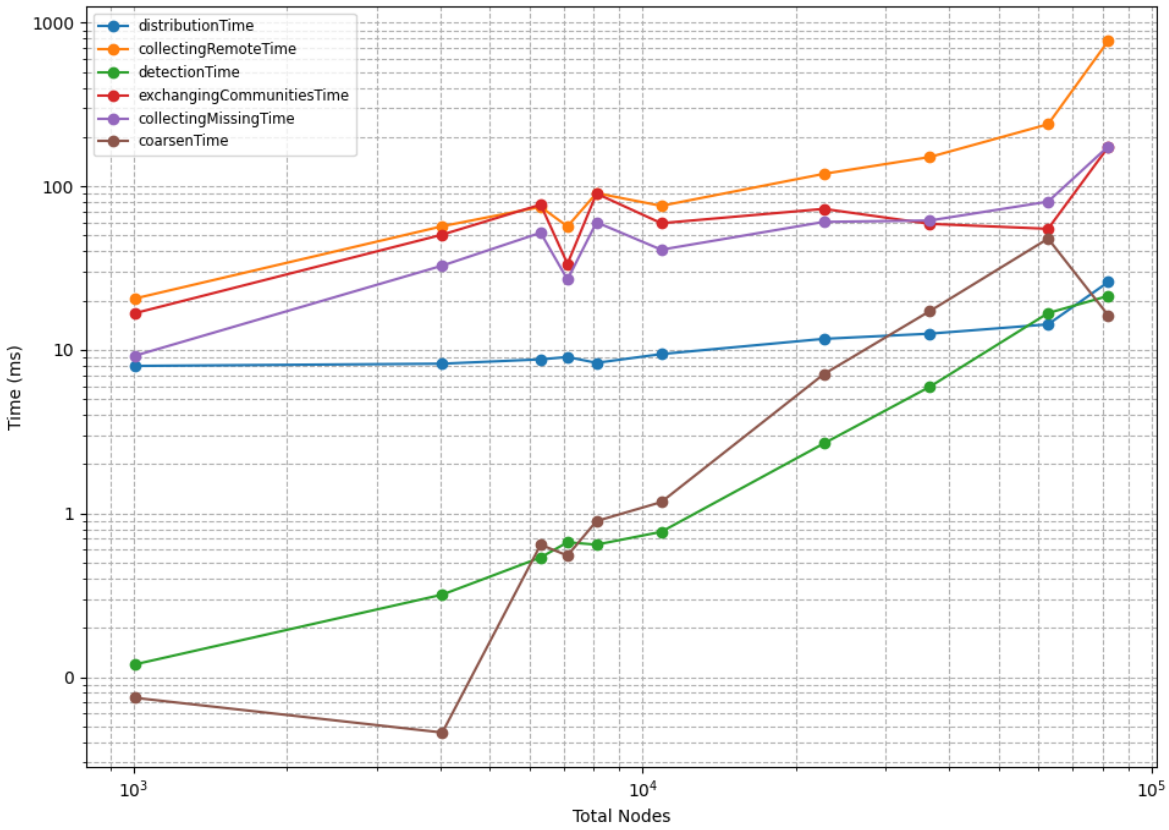

Figure 5.7: Time Breakdown against Total Nodes for all the dataset's graph and fixed size = 64

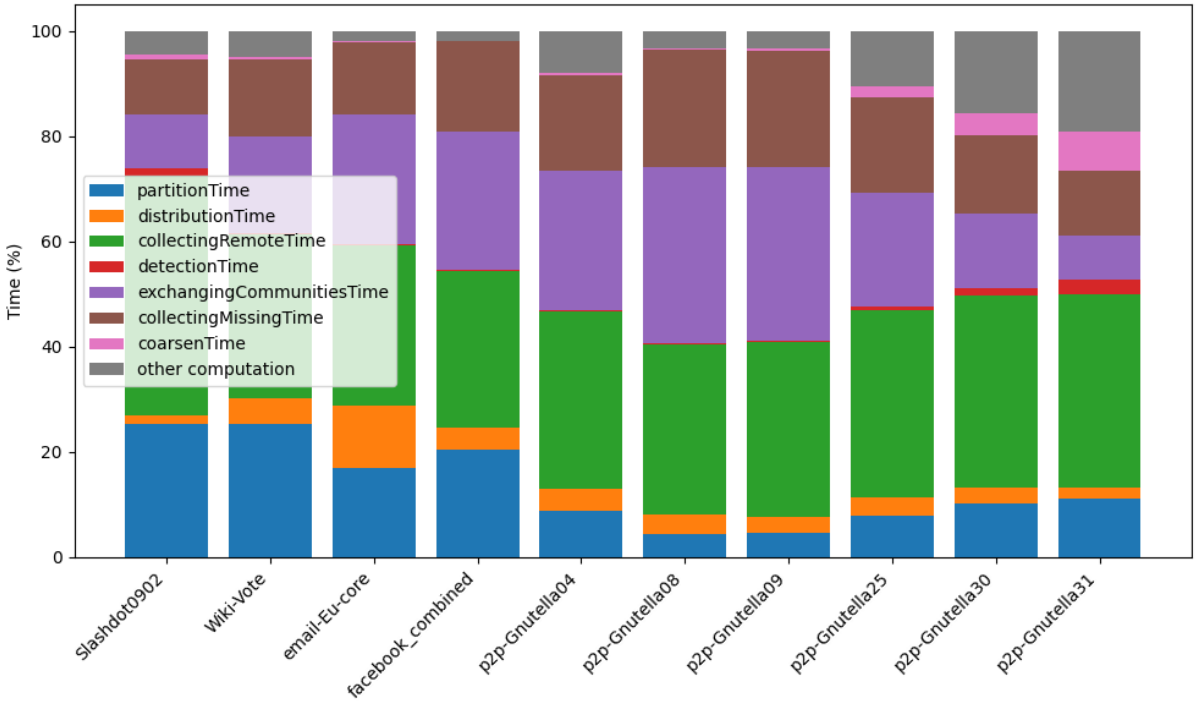

Figure 5.8: Time Breakdown by percentage for all the dataset's graph and fixed size = 64

Furthermore, the execution times of each step for each graph have been plotted (Figure 5.4, Figure 5.6 and Figure 5.8). These plots confirm the two previous observations. Firstly, increasing the number of processes decreases the computation time because each process has fewer nodes to assign. The communication time for remote nodes collection also decreases for this reason. However, the communication time for updates increases because more processes must circulate them. Secondly, as was observed before with the speedup, the communication time for denser graphs, especially for remote nodes collection, is greater than that required for sparser ones.

6 Conclusion and future work

6.1 Conclusion

The work presented a distributed parallel implementation of Rabbit Order [3] using MPI, with a particular focus on improving the performance of the community detection on the input graph. The present study is an investigation into the development of a scalable, distributed Louvain-based community detection algorithm. The algorithm has been designed to fully exploit MPI one-sided communication with remote memory access, and is based on the shared-memory parallel version of Rabbit Order and the distributed Louvain algorithm by Sattar et al. [40], we designed a scalable distributed Louvain-based community detection algorithm by fully exploiting MPI one-sided communication with remote memory access. Moreover, we eliminated the centralised redistribution and partitioning of the found communities by letting processes read the needed nodes for coarsening directly from each other.

The experimental results indicated promising performance and scalability under a range of conditions, particularly with larger graphs. Some configurations attained ideal speedup. Furthermore, the application produced better results with sparser graphs and more balanced networks, while communication overhead was revealed to be heavier for densely connected ones.

As anticipated, the main bottlenecks were found in increasing the number of processes, especially for smaller graphs. This shows the need for an improved partitioning system for load balancing, as well as the need to adjust the number of processes dynamically, especially in the final iterations of community detection.

6.2 Future work

Starting from the proposed implementation, it is possible to analyse potential improvements that could be applied to the algorithm:

- The key to straightforward performance improvement is a well-chosen partitioning function or approach for the initial and subsequent distribution of the graph, as the complexity analysis showed. Furthermore, distributed parallel partitioning could significantly reduce the preliminary steps required for this type of parallelisation.
- Another level of parallelisation can be introduced by using shared-memory techniques for computations within single processes, i.e. OpenMP techniques.

Overall, the proposed MPI-based reordering algorithm demonstrates that Rabbit Order can be effectively extended to distributed-memory systems, paving the way for further optimisations and broader applicability in large-scale graph analytics (e.g. Graph Neural Networks or Proteins Analysis).

Bibliography

[1] Cluster hpc - unitrento service desk. `https://servicedesk.unitn.it/go/it?s=S00111`. Last access 23/06/2025.

[2] Patrick R. Amestoy, Timothy A. Davis, and Iain S. Duff. An approximate minimum degree ordering algorithm. *SIAM Journal on Matrix Analysis and Applications*, 17(4):886–905, 1996.

[3] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Iwamura. Rabbit order: Just-in-time parallel reordering for fast graph analysis. In *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 22–31, 2016.

[4] Michael Bader. *Space-filling Curves. An Introduction With Applications in Scientific Computing*, volume 9. Springer Berlin Heidelberg, 01 2013.

[5] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, October 2008.

[6] Aydin Buluç, Jeremy T. Fineman, Matteo Frigo, John R. Gilbert, and Charles E. Leiserson. Parallel sparse matrix-vector and matrix-transpose-vector multiplication using compressed sparse blocks. In *Proceedings of the Twenty-First Annual Symposium on Parallelism in Algorithms and Architectures*, SPAA '09, page 233–244, New York, NY, USA, 2009. Association for Computing Machinery.

[7] E. Cuthill and J. McKee. Reducing the bandwidth of sparse symmetric matrices. In *Proceedings of the 1969 24th National Conference*, ACM '69, page 157–172, New York, NY, USA, 1969. Association for Computing Machinery.

[8] J. J. Dongarra, Jeremy Du Croz, Sven Hammarling, and I. S. Duff. A set of level 3 basic linear algebra subprograms. *ACM Trans. Math. Softw.*, 16(1):1–17, March 1990.

[9] ENCCS contributors. Intermediate mpi - intermediate mpi. `https://enccs.github.io/intermediate-mpi/`. Last access 23/06/2025.

[10] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.

[11] Santo Fortunato and Claudio Castellano. Community structure in graphs. In Robert A. Meyers, editor, *Computational Complexity: Theory, Techniques, and Applications*, pages 490–512. Springer New York, New York, NY, 2012.

[12] GeeksforGeeks contributors. Sparse matrix and its representations — set 2 (using list of lists and dictionary of keys). `https://www.geeksforgeeks.org/dsa/sparse-matrix-representations-using-list-lists-dictionary-keys/`. Last access 23/06/2025.

[13] Alan George. Nested dissection of a regular finite element mesh. *SIAM Journal on Numerical Analysis*, 10(2):345–363, 1973.

[14] GNU Project. Gcc online documentation – version 9.1.0. https://gcc.gnu.org/onlinedocs/9.1.0/. Last access 24/06/2025.

[15] Laure Gonnord, Ludovic Henrio, Lionel Morel, and Gabriel Radanne. A survey on parallelism and determinism. *ACM Comput. Surv.*, 55(10), February 2023.

[16] Kenneth M. Hall. An r-Dimensional Quadratic Placement Algorithm. *Management Science*, 17(3):219–229, November 1970.

[17] hpc-wiki.info contributors. Scaling – hpc wiki. https://hpc-wiki.info/hpc/Scaling. Last access 24/06/2025.

[18] George Karypis and Vipin Kumar. METIS — a software package for partitioning unstructured graphs, partitioning meshes and computing fill-reducing ordering of sparse matrices. *University of Minnesota*, 01 1997.

[19] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.

[20] William Kelly, Rob Johnson, and Wei Zhang. Graph representation matters: Optimizing performance on gpu. *USENIX ;login:*, 45(4):16–21, 2020.

[21] Jure Leskovec and Andrej Krevl. p2p-gnutella04. https://snap.stanford.edu/data/p2p-Gnutella04.html. Last access 23/06/2025.

[22] Jure Leskovec and Andrej Krevl. p2p-gnutella08. https://snap.stanford.edu/data/p2p-Gnutella08.html. Last access 23/06/2025.

[23] Jure Leskovec and Andrej Krevl. p2p-gnutella09. https://snap.stanford.edu/data/p2p-Gnutella09.html. Last access 23/06/2025.

[24] Jure Leskovec and Andrej Krevl. p2p-gnutella25. https://snap.stanford.edu/data/p2p-Gnutella25.html. Last access 23/06/2025.

[25] Jure Leskovec and Andrej Krevl. p2p-gnutella30. https://snap.stanford.edu/data/p2p-Gnutella30.html. Last access 23/06/2025.

[26] Jure Leskovec and Andrej Krevl. p2p-gnutella31. https://snap.stanford.edu/data/p2p-Gnutella31.html. Last access 23/06/2025.

[27] Jure Leskovec and Andrej Krevl. Soc-slashdot0902. https://snap.stanford.edu/data/soc-Slashdot0902.html. Last access 23/06/2025.

[28] Jure Leskovec and Andrej Krevl. Wiki-vote. https://snap.stanford.edu/data/wiki-Vote.html. Last access 23/06/2025.

[29] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. http://snap.stanford.edu/data, June 2014.

[30] Andrew Lumsdaine, Douglas Gregor, Bruce Hendrickson, and Jonathan Berry. Challenges in parallel graph processing. *Parallel Processing Letters*, 17:5–20, 03 2007.

[31] J. McAuley and J. Leskovec. Ego-facebook. https://snap.stanford.edu/data/ego-Facebook.html. Last access 23/06/2025.

[32] MPI Forum. Mpi forum. https://www.mpi-forum.org/. Last access 23/06/2025.

[33] MPI Forum. *MPI: A Message-Passing Interface Standard Version 3.1.* MPI Forum, 2015. Available at https://fs.hlrs.de/projects/par/mpi/mpi31/.

[34] M. E. J. Newman and M. Girvan. Finding and evaluating community structure in networks. *Phys. Rev. E*, 69:026113, Feb 2004.

[35] Giang Nguyen, Viera Šipková, Stefan Dlugolinsky, Binh Minh Nguyen, Viet Tran, and Ladislav Hluchý. A comparative study of operational engineering for environmental and compute-intensive applications. *Array*, 12:100096, 2021.

[36] NMSU High Performance Computing Team. Message passing interface :: High performance computing. `https://hpc.nmsu.edu/discovery/mpi/introduction/`. Last access 23/06/2025.

[37] Duane Q. Nykamp. An introduction to networks - math insight. `https://mathinsight.org/network_introduction`. Last access 23/06/2025.

[38] Sasko Ristov, Radu Prodan, Marjan Gusev, and Karolj Skala. Superlinear speedup in hpc systems: Why and when? In *2016 Federated Conference on Computer Science and Information Systems (FedCSIS)*, pages 889–898, 2016.

[39] Benjamin Sanchez-Lengeling, Emily Reif, Adam Pearce, and Alexander B. Wiltschko. A gentle introduction to graph neural networks. *Distill*, 2021. https://distill.pub/2021/gnn-intro.

[40] Shaikh Sattar, Naw Safrinand Arifuzzaman. Scalable distributed louvain algorithm for community detection in large graphs. *The Journal of Supercomputing*, 78(7):10275–10309, May 2022.

[41] Hiroaki Shiokawa, Yasuhiro Fujiwara, and Makoto Onizuka. Fast algorithm for modularity-based graph clustering. In *Proceedings of the Twenty-Seventh AAAI Conference on Artificial Intelligence*, AAAI'13, page 1170–1176. AAAI Press, 2013.

[42] Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.

[43] MPICH Team. Mpich version 3.2.1. `https://www.mpich.org/static/downloads/3.2.1/`. Last access 24/06/2025.

[44] Wikipedia contributors. Adjacency matrix - wikipedia. `https://en.wikipedia.org/wiki/Adjacency_matrix`. Last access 23/06/2025.

[45] Wikipedia contributors. Sparse matrix - wikipedia. `https://en.wikipedia.org/wiki/Sparse_matrix`. Last access 23/06/2025.

[46] Hao Yin, Austin R. Benson, Jure Leskovec, and David F. Gleich. Email-eu-core. `https://snap.stanford.edu/data/email-Eu-core.html`. Last access 23/06/2025.

[47] Raphael Yuster and Uri Zwick. Fast sparse matrix multiplication. *ACM Trans. Algorithms*, 1(1):2–13, July 2005.

Appendix A Notations

Table A.1: List of notations and variable names

Name	**Meaning**
G	Input graph
V	Set of all graph nodes
E	Set of all graph edges
n	Order of input graph $\|V\|$
m	Size of input graph $\|E\|$
$deg(v)$	Degree of node v
Q	Modularity of graph
$\Delta(u, v)$	Modularity gain for inserting u in community of v
P	Set of all processes
$size$	Number of processes $\|P\|$
$rank$	Identifier of a process
P_i	Process with $rank\ i$
P_0	Root process
$p2n : P \to \mathcal{P}(V)$	Mapping for (process, assigned nodes)
$p2e : P \to \mathcal{P}(E)$	Mapping for (process, assigned edges)
$n2p : V \to P$	Partition mapping for (node, assigned process)
$localNodes \subseteq V$	Set of collected assigned nodes
$remoteNodes \subseteq V$	Set of collected remote nodes
$collectedNodes = localNodes \cup remoteNodes$	Set of all collected nodes
$neighbours : collectedNodes \to \mathcal{P}(V)$	Adjacency list of collected nodes
$n2c : V \to V$	Mapping for (node, assigned community)
$tot : V \to \mathbb{N}$	Mapping for (community, total weight)
$in : V \to \mathbb{N}$	Mapping for (community, internal weight)
$c2n : V \to \mathcal{P}(V)$	Mapping for (community, assigned nodes)

“Knock knock”
“Race condition”
“Who’s there?”